Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 7

Cài đặt và thực thi code thuật toán Hierarchic:

- Import các thư viện và load dữ liệu:

```
import numpy as np
import pandas as pd
import seaborn as sns
import scipy.cluster.hierarchy as shc
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering

customer_data = pd.read_csv(r"D:\data_mining\tuan7\data.csv")
customer_data
```

✓ 0.0s

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

- Lấy dữ liệu cần xử lý (ở đây là annual income và spending score):

```
selected_data = customer_data.iloc[:, 3:5]
selected_data
```

✓ 0.0s

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40
...	...	...
195	120	79
196	126	28
197	126	74
198	137	18
199	137	83

200 rows × 2 columns

- Tạo hàm tính khoảng cách Euclid từ 2 tập điểm và phương pháp sử dụng. Ở đây X có dạng [X1.x, X1.y, X2.x, X2.y, …, Xn.x, Xn.y] tượng trưng cho đã ghép n điểm và X, tương tự với Y. Hàm sẽ tính khoảng cách Euclid giữa toàn bộ điểm trong X tới toàn bộ điểm trong Y và dựa trên phương pháp đang sử dụng mà trả về kết quả cần. Nếu phương pháp đang sử dụng là “single” thì trả về min khoảng cách X và Y, “complete” thì trả về max khoảng cách X và Y, còn không thì trả về average khoảng cách X và Y:

```python
def Euclidean(X, Y, method):
    Dist = []
    for i in range(0, len(X), 2):
        for j in range(0, len(Y), 2):
            distance = (X[i] - Y[j]) ** 2 + (X[i + 1] - Y[j + 1]) ** 2
            Dist.append(np.sqrt(distance))
    if method == "single":
        return np.min(Dist)
    if method == "complete":
        return np.max(Dist)
    return np.average(Dist)
```

- Tạo hàm chạy thuật toán hierarchic với các tham số là data (dữ liệu sử dụng), n_cluster (số tập cuối cùng sau khi thực thi thuật toán) và method (cách làm thuật toán, có thể là "single", "complete" và "average").
  Trong hàm này đầu tiên nó sẽ chuyển data về dạng điểm để dễ xử lý.
  Sau đó duyệt các cặp tập điểm, với mỗi cặp như vậy, tìm khoảng cách nhỏ nhất của 2 tập điểm gọi là indx và indy. Sau đó gộp tập điểm indx và indy vào làm 1. Lặp lại các bước trên cho đến khi số tập điểm còn chính xác là n_cluster.
  Hàm sẽ trả về n_cluster tập điểm:

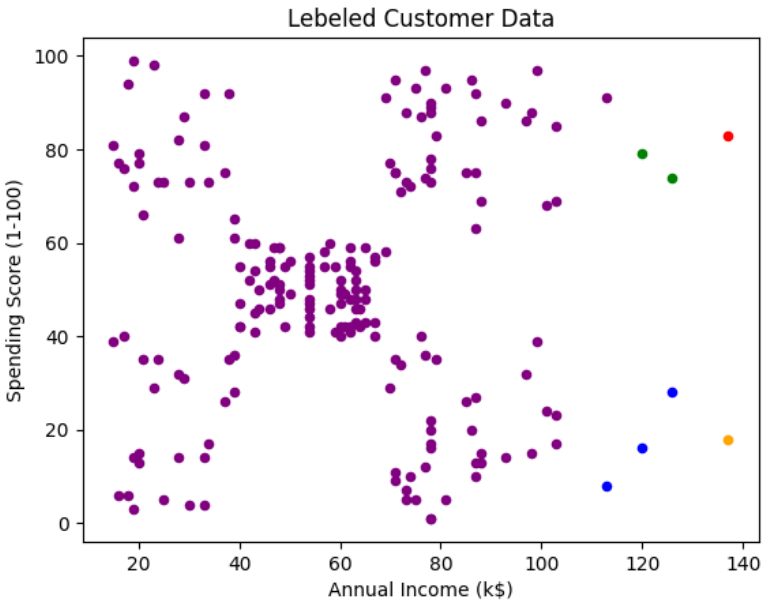
```python
def hierarchic(data, n_cluster, method):
    Points = []
    for i in range(data.shape[0]):
        Points.append([int(data.iloc[i, 0]), int(data.iloc[i, 1])])

    n = len(Points)

    while(n > n_cluster):
        Min = float('inf')
        for i in range(len(Points)):
            for j in range(i + 1, len(Points)):
                dist = Euclidean(Points[i], Points[j], method=method)
                if dist < Min:
                    Min = dist
                    indx, indy = i, j

        for k in range(0, len(Points[indy])):
            Points[indx].append(Points[indy][k])
        Points.pop(indy)

        n -= 1

    return Points
```

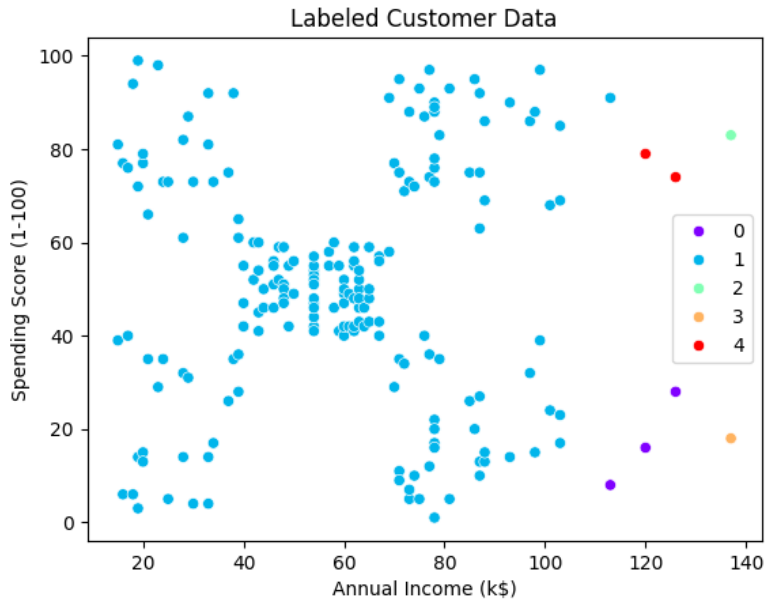
So sánh kết quả thuật toán Hierarchic tự code và code có sẵn:

Sau khi có được tập Points thì ta sẽ vẽ từng tập điểm trong Points với 1 màu khác nhau xem như 1 nhóm:

```
Points = hierarchic(selected_data, 5, method="single")
colors = ['purple', 'blue', 'green', 'orange', 'red']

for i in range(len(Points)):
    current_cluster = Points[i]
    x = [current_cluster[j] for j in range(0, len(current_cluster), 2)]
    y = [current_cluster[j] for j in range(1, len(current_cluster), 2)]

    plt.scatter(x, y, s=20, c=colors[i], label=f'Cluster {i+1} ({len(x)} điểm)')

plt.title('Lebeled Customer Data')

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.show()
```

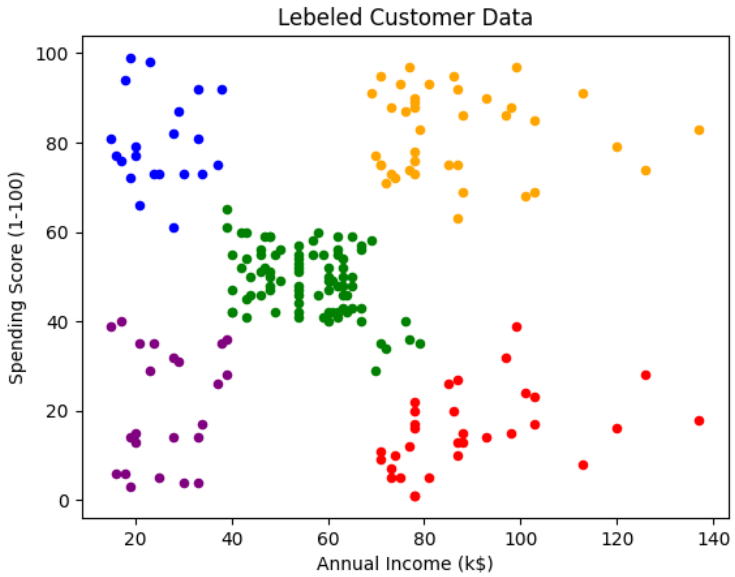
- Với phương pháp single tự code và code có sẵn:

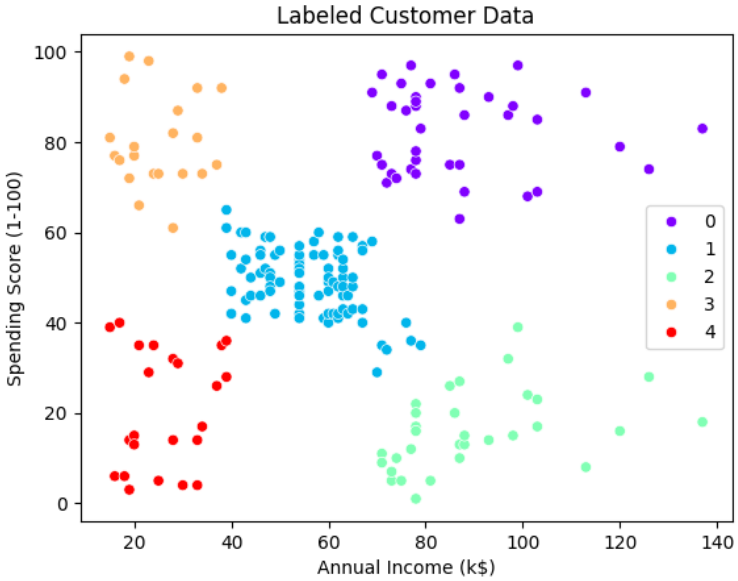




⇨ Giống nhau

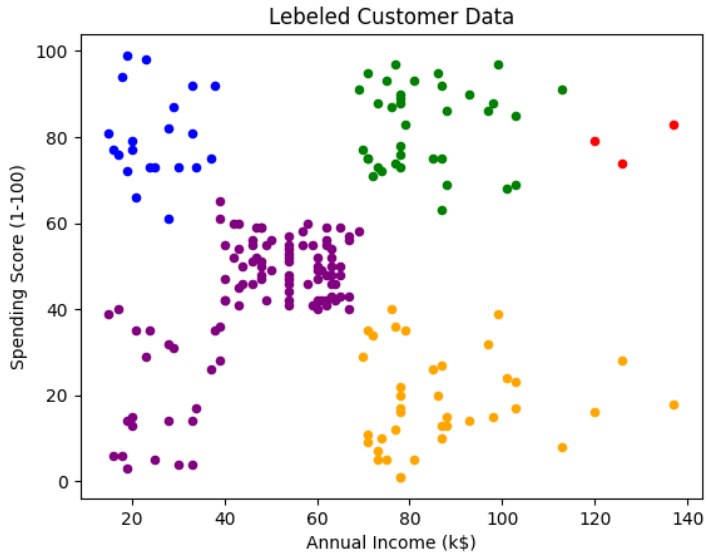
- Với phương pháp complete tự code và code có sẵn:

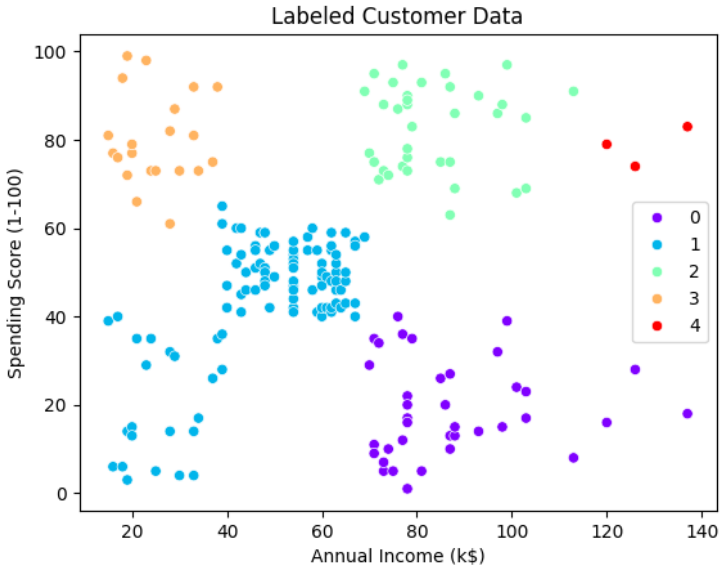




⇨ Giống nhau

Với phương pháp average tự code và code có sẵn:





⇨ Giống nhau

Vậy code thuật toán Hierarchic tự code đúng so với code có sẵn: